RHI Architectural Evaluation for Modern APIs

https://gemini.google.com/app/091167ff7e6a3cfe

User prompt: I want you to check my render/ architecture and its soundness. Im using modern opengl currently but i have an abstract RHI which should support opengl, vulkan and d3s12. dont be nitpicky about small things but the broader architecture

Response: Your RHI foundation—using compile-time backend selection (`using CommandBuffer = GLCommandBuffer`) rather than heavy runtime virtual interfaces—is a performant, zero-overhead choice. However, the current abstraction model closely mirrors OpenGL's immediate, stateful execution model, which introduces several severe architectural roadblocks when targeting Vulkan and DirectX 12.

Here is an architectural evaluation of where the design holds up, where it will break under explicit modern APIs, and how to align it.

1. Resource Binding & The Descriptor Set Race Condition

This is the most critical architectural issue in the engine right now.

In `MainOpaquePass::execute`, you mutate and bind descriptor sets inside the per-mesh draw loop:

```
pass_descriptor_set_->write_texture(2, *albedo);
cb.bind_descriptor_set(*pass_descriptor_set_);
cb.draw_indexed(...);
```

- **Why it works in OpenGL:** `GLDescriptorSet::bind()` executes `glBindTextureUnit` immediately on the CPU. The driver updates its current active texture unit state before issuing the draw call.
- **Why it breaks in Vulkan and DX12:** Command buffers record commands for deferred execution on the GPU. Descriptor sets in Vulkan (`VkDescriptorSet`) and descriptor tables in DX12 are backed by GPU-visible descriptor pools/heaps. If you overwrite slot 2 of `pass_descriptor_set_` on the CPU while recording, you are mutating the exact same descriptor memory. By the time the GPU actually executes the command buffer at submit time, **every single draw call will sample whichever texture was written during the very last loop iteration**.
- **Missing Set / Layout Indices:** `cb.bind_descriptor_set(set)` takes no set index or pipeline layout. Vulkan's `vkCmdBindDescriptorSets` requires `(VkPipelineLayout, firstSet, descriptorSetCount, pDescriptorSets)`. DX12 requires the root parameter index defined in the Root Signature.
- **The Architectural Fix:** Do not update descriptor sets per draw call. You already have the architectural solution half-built in `render/gpu/`:
    1. **Bindless / Descriptor Indexing:** Place all textures into an unbounded descriptor array (`Texture2D textures[]` in shaders).
    2. **Material Data in SSBO:** Store texture indices in `GPUMaterialData` (as you designed: `albedo_texture_index`).

3. Draw calls then only pass a material/instance ID (via `gl_BaseInstance`, push constants, or an instance SSBO). The descriptor set is bound **once per pass**, completely eliminating mid-pass descriptor mutations.

2. PSO Creation & Framebuffer Format Coupling

In OpenGL, shader programs and VAOs are decoupled from framebuffers; you can bind any shader to any render target regardless of its pixel format.

In Vulkan and DX12, a Pipeline State Object (PSO) is strictly immutable and baked against the target render pass:

- **DX12:** `D3D12_GRAPHICS_PIPELINE_STATE_DESC` requires `RTVFormats` (e.g., `DXGI_FORMAT_R8G8B8A8_UNORM`) and `DSVFormat`.
- **Vulkan (Dynamic Rendering / 1.3):** `VkGraphicsPipelineCreateInfo` requires `VkPipelineRenderingCreateInfo` with the color attachment formats and depth attachment format.

Currently, `PipelineStateDesc` only contains shaders, vertex layout, rasterizer, depth, and blend states:

```
struct PipelineStateDesc {
    Shared<Shader> shader{nullptr};
    BufferLayout vertex_layout;
    // ... cull, depth, blend ...
    // MISSING: Target Framebuffer / Render Target attachment formats!
};
```

If your RHI cannot query target attachment formats at pipeline creation time, you will be forced to compile PSOs lazily during `cb.begin_render_pass()` or `cb.draw()`, causing severe runtime micro-stuttering whenever a new material/render target combination appears.

The Fix: Add the expected render target formats to `PipelineStateDesc`:

```
Vector<TextureFormat> color_formats;
TextureFormat depth_format = TextureFormat::None;
```

3. Queue Submission & Explicit Resource Synchronization

Modern low-level APIs treat command recording, submission, and frame presentation as distinct asynchronous stages. Your RHI currently lacks three fundamental primitives:

Queue Concept & Submission

In `RenderContext`:

```
void begin_frame() { command_buffers_[current_frame_index_]->begin(); }
void end_frame() {
    command_buffers_[current_frame_index_]->end();
    swapchain_->present();
    current_frame_index_ = (current_frame_index_ + 1) % MAX_FRAMES_IN_FLIGHT;
}
```

In Vulkan/DX12, calling `end()` on a command buffer does nothing on the GPU. You must submit command buffers to a queue (`VkQueue` / `ID3D12CommandQueue`):

```
Record -> Queue Submit (with Wait/Signal Semaphores) -> Present
```

Without an explicit `submit(CommandBuffer&)` step in the RHI, you cannot control queue synchronization or multi-queue execution (e.g., async compute).

Resource Transitions and Layout Barriers

`GLCommandBuffer::memory_barrier(MemoryBarrierFlags flags)` exposes OpenGL's `glMemoryBarrier`. Vulkan and DX12 require tracking resource states:

- Textures must transition between states (e.g., `UNDEFINED` → `COLOR_ATTACHMENT_OPTIMAL` → `SHADER_READ_ONLY_OPTIMAL` → `PRESENT_SRC_KHR` or `D3D12_RESOURCE_STATE_RENDER_TARGET` → `D3D12_RESOURCE_STATE_PIXEL_SHADER_RESOURCE`).
- This is where your `RenderGraph` should take over: the graph should declare read/write dependencies and emit backend-specific barrier commands (`VkImageMemoryBarrier2` / `D3D12_RESOURCE_BARRIER`) rather than requiring manual barriers in passes.

4. Multi-Buffering and the CPU-GPU Pipeline Split

Per-Frame Resource Double-Buffering

`RenderContext` sets `MAX_FRAMES_IN_FLIGHT = 2`, but the backing resources are singletons:

- `frame_ubo_` in `SceneRenderer` is a single buffer updated every frame via `set_data()`.
- `object_ssbo_` in `MainOpaquePass` is a single buffer updated every frame.

While GPU Frame 0 is executing, the CPU starts recording Frame 1. Overwriting `frame_ubo_` or `object_ssbo_` from the CPU while the GPU is reading from it causes a race condition (tearing/flickering). Any CPU-writable buffer must either be ring-buffered by `MAX_FRAMES_IN_FLIGHT` or allocated out of a transient per-frame upload buffer. Additionally, `RenderContext::begin_frame()` must await a CPU-GPU fence for the active frame index before recording into it.

Resolving the Two Architectures

There is currently an architectural divergence in the codebase:

1. **The Classic Pipeline (`SceneRenderer` & `MainOpaquePass`):** Per-mesh VBO/IBOs, CPU submission loop, descriptor rebinding per mesh, individual draw calls.
2. **The GPU-Driven Pipeline (`render/gpu/`):** `GPUMeshManager` (megabuffers), `MaterialSystem`, `TextureSystem`, and `GPUCuller` (`glMultiDrawElementsIndirect`).

The GPU-driven approach in `render/gpu/` is fundamentally more compatible with Vulkan and DX12 than the classic path. When you use global mega-buffers (`global_vbo_`, `global_ibo_`) and indirect multidraw:

- Vertex and index buffers are bound **once** per frame.
- Descriptor sets are bound **once** per pass.
- The CPU never loops over thousands of draw calls or mutates descriptor sets mid-flight.

UI Subsystem Decoupling

`RmlRenderInterface.cpp` and `UISystem.cpp` make direct calls to raw OpenGL (`glGenVertexArrays`, `glBufferData`, `glScissor`, `glTexImage2D`). This breaks the abstraction barrier entirely: compiling the engine with `RENDER_API=VULKAN` or `RENDER_API=DX12` will fail to compile or link. Refactor `RmlRenderInterface` to allocate its geometry via `RHIDevice::create_vertex_buffer()` and record UI draws into `CommandBuffer` just like the scene renderer.
